

Sets in Python:

A set is an unordered collection of unique elements. Common uses of sets include membership testing, removing duplicates from a sequence, and performing standard mathematical operations such as union, intersection, difference, and symmetric difference.

A set can be created by placing elements inside curly braces, similar to a dictionary but without key–value pairs:

```
S = {number1, number2, number3}
```

What makes a set particularly useful is the following:

- * Only unique elements are stored (duplicate values are automatically removed)**
- * Sets are mutable, meaning elements can be added or removed after creation**
- * Sets are unordered collections**
- * Sets support mathematical operations**
- * Sets use hashing internally**
- * Sets are mutable but Set elements must be immutable (hashable)**

Example, If we create a set,

```
>> S= {1, 2, 3, 3, 5, 1, 7, 8}  
>> print(S)
```

Shows, {1, 2, 3, 5, 7, 8} duplicate elements are automatically removed.

It must be noted that the order of the elements is not guaranteed because set is an unordered collection of elements.

Important note: `>> S = {}` [will create an empty dictionary not empty set]

`>> S = set()` [will create an empty set]

Type Casting:

A set can be created from many iterable data types using the set function.

(i) List $\longrightarrow$ Set

A set can also be created using iterables like a list.

```
>> S = set([1, 3, 5, 1, 5, 7])
>> print(S)                              #{1, 3, 5, 7}
```

(ii) Tuples $\longrightarrow$ Set

```
>> S = set((1, 2, 2, 3, 5, 6, 3, 4))
>> print(S)                              # {1, 6, 3, 4, 5, 2}
```

(iii) String $\longrightarrow$ Set

```
>> S = set("Apple")
>> print(S)                              # {'A', 'p', 'l', 'e'}
```

```
>> S = set("AappleE")
>> print(S)                                # {'A', 'E', 'p', 'l', 'e',
                                           'a'}
```

(iv) Range $\longrightarrow$ Set

```
>> S = set(range(1, 4))
>> print(S)                                # {1, 2, 3}
```

(v) Dictionary $\longrightarrow$ Set

```
>> S = set({"a": 1, "b": 2})
>> print(S)                                # {'b', 'a'}
```

Frozen set:

The `frozenset()` function in Python creates an immutable set from an iterable. It stores only unique values and does not allow modification after creation. Since it is immutable, a frozenset can be used as a dictionary key or as an element of another set.

```
>> a = frozenset([1, 2, 3])
>> print(a)                                # frozenset({1, 2, 3})
```

Methods in set:

(a) Common set methods:

Let `s = {1, 2, 3, 4, 5}`

(i) add()

```
>> s.add(element)      [Adds element to set "s"]
```

```
>> s.add(111)
```

```
>> print(s)            # {1, 2, 3, 4, 5, 111}
```

Note: add() will take only one element

(ii) update()

```
>> s.update(iterable)   [Adds multiple items to set "s"]
```

```
>> s.update([111,222])
```

```
>> print(s)            # {1, 2, 3, 4, 5, 222, 111}
```

Note: update() can have many iterables of various types such as list, tuple, range(), set etc.

```
>> s.update([11,22], (333, 444), {5555, 6666})
```

```
>> print(s)            # {1, 5, 3, 22, 5555, 333, 6666, 2, 4, 11,
                        444}
```

(iii) remove()

```
>> s.remove('x')        [Removes the item 'x' from the set "s"]
```

```
>> s.remove(3)
```

```
>> print(s)            # {1, 2, 4, 5}
```

(iv) discard()

```
>> s.discard('x')          [Removes the item 'x' from the set set "s"]
```

```
>> s.discard(3)
```

```
>> print(s)                #{1, 2, 4, 5}
```

- Note: `remove()` and `discard` are distinct.
- `remove()` raises an error if 'x' is not present in the set, whereas `discard()` does not raise any error (safe to use).

(v) pop()

```
>> s.pop()                 [Removes an arbitrary element from the set  
                           "s"]
```

```
>> print(s.pop())          # 1 [Output may vary]
```

```
>> print(s)                # {2, 3, 4, 5}
```

Note: `pop()` removes arbitrary element and in this case it is 1.

(vi) clear()

```
>> s.clear()               [Removes all elements from the set]
```

```
>> s.clear()
```

```
>> print(s)                # set()
```

(vii) copy()

```
>> s.copy()                [creates a new set object containing  
                           the same elements]
```

```
>> copy_s = s.copy()
```

```
>> print(copy_s)           # {1, 2, 3, 4, 5}
```

(b) Set Operation methods:

The following methods are commonly used for set operations and data manipulation.

Let us, $a = \{1, 2, 3, 4\}$, $b = \{3, 4, 5, 6\}$, $c = \{5, 6, 7, 8\}$, $d = \{1, 2, 3, 4, 5, 6\}$

- As in the list and dictionary, **len(a)** will give the number of elements present in the set.
- If x is any data and s is the given set, "**x in s**" will return True if present

```
>> 2 in a
```

Gives the result True

```
>> 5 in a
```

Gives the result False

- If x is any data and s is the given set, "**x not in s**" will return True if not present

```
>> 5 not in a
```

Gives the result True

```
>> 1 not in a
```

Gives the result False

- **S1.isdisjoint(S2)** returns True or False. If the given sets have no common element(not even one) then True is returned otherwise False

```
>> a.isdisjoint(c)
```

Gives True

```
>> a.isdisjoint(b)
```

Gives False

- **S1.issubset(S2)** returns True or False. If the set S2 has all the elements that are members of S1, then True is returned otherwise False

```
>> a.issubset(d)
```

Gives True because all elements (1, 2, 3, 4) of set 'a' in set 'd'

```
>> a.issubset(b)
```

Gives False because elements of the set 'a' not fully present in the set 'b' i.e., only 3, 4 are present 1, 2 not present.

- **S1.issuperset(S2)** returns True or False. If the set S1 has all the elements that are members of S2, then True is returned otherwise False

```
>> d.issuperset(a)
```

Gives True because all elements (1, 2, 3, 4) of set 'a' in set 'd'

```
>> a.issuperset(b)
```

Gives False because the set 'a' does not contain every element in the set 'b'

The following methods perform standard mathematical set operations.

Let S1 and S2 be sets.

- **S1.union(S2)** → Returns a new set which contains elements of S1 as well as elements of S2.

```
>> a.union(b)
Gives {1, 2, 3, 4, 5, 6}
```

union() function can contain multiple sets in which case, the resulting set will contain all the elements of the given sets.

```
>> a.union(b, c)
Gives {1, 2, 3, 4, 5, 6, 7, 8}
```

- S1.**intersection**(S2) → Returns a new set which contains elements that are common to both S1 as well as S2.

```
>> a.intersection(b)
Gives {3, 4}
```

intersection() function can contain multiple sets in which case, the resulting set contains the elements that are common to all sets involved in the operation.

```
>> b.intersection(c, d)
Gives {5, 6}
```

S1.**difference**(S2) → Returns a new set which contains elements that are members of S1 after removing elements which are members of S2.

```
>> a.difference(b)
Gives {1, 2}
```

difference() function can contain multiple sets.

- S1.**symmetric_difference**(S2) → Returns a new set which contains elements that are in either S1 or S2 but not both.

```
>> a.symmetric_difference(b)
Gives {1, 2, 5, 6}
```

symmetric_difference() accepts exactly one argument at a time.

Method	Multiple Arguments Allowed?
union()	Yes
intersection()	Yes
difference()	Yes
Symmetric_difference	No

Please note that in place of union(), intersection(), difference(), symmetric_difference(), we can use operators as provided in the below table.

Set Operations: Methods and Operators

Operation	Method	Operator
Union	a.union(b)	$a \mid b$
Intersection	a.intersection(b)	$a \& b$
Difference	a.difference(b)	$a - b$
Symmetric Difference	a.symmetric_difference(b)	$a \wedge b$

Special update methods:

- **intersection_update():**

```
>> a.intersection_update(b)
>> print(a)                # {3, 4}
```

- **difference_update():**

```
>> a.difference_update(b)
>> print(a)                #{1, 2}
```

- **symmetric_difference_update():**

```
>> a.symmetric_difference_update(b)
```

```
>> print(a)                # {1, 2, 5, 6}
```

There is no union_update() because update() itself is the short form of union_update().

Operators for special update methods

Update Method	Operator
update()	=
intersection_update()	&=
difference_update()	-=
symmetric_difference_update()	^=

Comparison of Various data types:

Type	Ordered	Mutable	Duplicate Allowed
List	Yes	Yes	Yes
Tuple	Yes	No	Yes
Set	No	Yes	No
Frozenset	No	No	No